

27/04/2026

What is Python?

Python is a popular programming language. It was created by Guido van Rossum, and released in 1991.

It is used for:

- web development (server-side),
- software development,
- mathematics,
- system scripting.

What can Python do?

- Python can be used on a server to create web applications.
- Python can be used alongside software to create workflows.
- Python can connect to database systems. It can also read and modify files.
- Python can be used to handle big data and perform complex mathematics.
- Python can be used for rapid prototyping, or for production-ready software development.

Why Python?

- Python works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- Python has a simple syntax similar to the English language.
- Python has syntax that allows developers to write programs with fewer lines than some other programming languages.
- Python runs on an interpreter system, meaning that code can be executed as soon as it is written. This means that prototyping can be very quick.
- Python can be treated in a procedural way, an object-oriented way or a functional way.

Good to know

- The most recent major version of Python is Python 3, which we shall be using in this tutorial.
- In this tutorial Python will be written in a text editor. It is possible to write Python in an Integrated Development Environment, such as Thonny, Pycharm, Netbeans or Eclipse which are particularly useful when managing larger collections of Python files.

Python Syntax compared to other programming languages

- Python was designed for readability, and has some similarities to the English language with influence from mathematics.
- Python uses new lines to complete a command, as opposed to other programming languages which often use semicolons or parentheses.

- Python relies on indentation, using whitespace, to define scope; such as the scope of loops, functions and classes. Other programming languages often use curly-brackets for this purpose.

Python Indentation

Indentation refers to the spaces at the beginning of a code line.

Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to indicate a block of code.

Example:

```
if 5 > 2:
    print("Five is greater than two!")
```

Python will give you an error if you skip the indentation

Python Variables

In Python, a variable is created when you assign a value to it:

```
x = 5
y = "Hello, World!"
```

Python has no command for declaring a variable.

Comments

Python has commenting capability for the purpose of in-code documentation.

Comments start with a #, and Python will render the rest of the line as a comment:

Example

Comments in Python:

```
#This is a comment.
print("Hello, World!")
```

Statements

A **computer program** is a list of "instructions" to be "executed" by a computer.

In a programming language, these programming instructions are called **statements**.

The following statement prints the text "Python is fun!" to the screen:

In Python, a statement usually ends when the line ends. You do *not* need to use a semicolon (;) like in many other programming languages (for example, [Java](#) or [C](#)).

Many Statements

Most Python programs contain many statements.

The statements are executed one by one, in the same order as they are written:

```
print("Hello World!")  
print("Have a good day.")  
print("Learning Python is fun!")
```

Semicolons (Optional, Rarely Used)

Semicolons are optional in Python. You can write multiple statements on one line by separating them with ; but this is rarely used because it makes it hard to read:

Print Text

you can use the [print\(\)](#) function as many times as you want. Each call prints text on a new line by default

Double Quotes

Text in Python must be inside quotes. You can use either " double quotes or ' single quotes:

Print Without a New Line

By default, the [print\(\)](#) function ends with a new line.

If you want to print multiple words on the same line, you can use the `end` parameter:

```
print("Hello World!", end=" ")
print("I will print on the same line.")
```

Note that we add a space after `end=" "` for better readability.

Print Numbers

You can also use the `print()` function to display numbers:

However, unlike text, we don't put numbers inside double quotes:

You can also do math inside the `print()` function:

```
print("I am", 35, "years old.")
```

Python Comments

Comments can be used to explain Python code.

Comments can be used to make the code more readable.

Comments can be used to prevent execution when testing code.

Creating a Comment

Comments starts with a `#`, and Python will ignore them:

```
#This is a comment
print("Hello, World!")
```

Multiline Comments

Python does not really have a syntax for multiline comments.

To add a multiline comment you could insert a `#` for each line:

```
#This is a comment
#written in
#more than just one line
"""
This is a comment
written in
more than just one line
"""
```

Variables

Variables are containers for storing data values.

Creating Variables

Python has no command for declaring a variable.

A variable is created the moment you first assign a value to it.

```
x = 5
y = "John"
print(x)
print(y)
```

Variables do not need to be declared with any particular *type*, and can even change type after they have been set.

```
x = 4          # x is of type int
x = "Sally"    # x is now of type str
print(x)
```

Casting

If you want to specify the data type of a variable, this can be done with casting.

```
x = str(3)      # x will be '3'
y = int(3)      # y will be 3
z = float(3)    # z will be 3.0
```

Get the Type

You can get the data type of a variable with the [`type\(\)`](#) function.

```
x = 5
y = "John"
print(type(x))
print(type(y))
```

Single or Double Quotes?

String variables can be declared either by using single or double quotes:

Case-Sensitive

Variable names are case-sensitive.

```
a = 4
A = "Sally"
#A will not overwrite a
```

Variable Names

A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume).

Rules for Python variables:

- A variable name must start with a letter or the underscore character
- A variable name cannot start with a number
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _)
- Variable names are case-sensitive (age, Age and AGE are three different variables)
- A variable name cannot be any of the [Python keywords](#).

```
myvar = "John"
my_var = "John"
_my_var = "John"
myVar = "John"
MYVAR = "John"
myvar2 = "John"
```

Multi Words Variable Names

Variable names with more than one word can be difficult to read.

There are several techniques you can use to make them more readable:

Camel Case

Each word, except the first, starts with a capital letter:

```
myVariableName = "John"
```

Pascal Case

Each word starts with a capital letter:

```
MyVariableName = "John"
```

Snake Case

Each word is separated by an underscore character:

```
my_variable_name = "John"
```

Many Values to Multiple Variables

Python allows you to assign values to multiple variables in one line:

```
x, y, z = "Orange", "Banana", "Cherry"
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

One Value to Multiple Variables

And you can assign the *same* value to multiple variables in one line:

```
x = y = z = "Orange"
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

Unpack a Collection

If you have a collection of values in a [list](#), [tuple](#) etc. Python allows you to extract the values into variables. This is called *unpacking*.

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x)
print(y)
print(z)
```

Output Variables

The `print()` function is often used to output variables.

```
x = "Python is awesome"
print(x)
x = "Python"
y = "is"
z = "awesome"
print(x, y, z)
x = "Python "
y = "is "
z = "awesome"
print(x + y + z)
```

Global Variables

Variables that are created outside of a function (as in all of the examples in the previous pages) are known as global variables.

Global variables can be used by everyone, both inside of functions and outside.

```
x = "awesome"

def myfunc():
    print("Python is " + x)

myfunc()
```

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.

```
x = "awesome"

def myfunc():
    x = "fantastic"
    print("Python is " + x)

myfunc()

print("Python is " +
```


The global Keyword

Normally, when you create a variable inside a function, that variable is local, and can only be used inside that function.

To create a global variable inside a function, you can use the [global](#) keyword.

```
def myfunc():  
    global x  
    x = "fantastic"
```

```
myfunc()
```

```
print("Python is " + x)
```

To change the value of a global variable inside a function, refer to the variable by using the [global](#) keyword:

```
x = "awesome"
```

```
def myfunc():  
    global x  
    x = "fantastic"
```

```
myfunc()
```

```
print("Python is " + x)
```

PYTHON DATA TYPES (EASY THEORY + EXAMPLES)

In programming, data type is an important concept.

Variables can store data of different types, and different types can do different things.

Python has the following data types built-in by default, in these categories:

Text Type: `str`

Numeric Types: `int, float, complex`

Sequence Types: `list, tuple, range`

Mapping Type: `dict`

Set Types: `set, frozenset`

Boolean Type: `bool`

Binary Types: `bytes, bytearray, memoryview`

None Type: `NoneType`

1. Numeric Types → store numbers

`x = 10` # int → whole numbers

`y = 3.14` # float → decimal numbers

`z = 2 + 3j` # complex → real + imaginary

2. Sequence Types → ordered collection (index-based)

`name = "Alice"` # str → text (immutable)

`fruits = ["apple", "banana"]` # list → changeable (mutable)

`coords = (10, 20)` # tuple → fixed (immutable)

`nums = range(5)` # range → sequence of numbers (0–4)

3. Set Types → unordered, no duplicates

`s = {1, 2, 3}` # set → mutable

`fs = frozenset([1, 2, 3])` # frozenset → immutable

4. Mapping Type → key-value pairs

`person = {"name": "Alice", "age": 25}` # dict → fast lookup using keys

5. Boolean Type → True or False

`is_valid = True` # bool → used in conditions

6. Binary Types → raw byte data

`b = b"hello"` # bytes → immutable

`ba = bytearray([65,66,67])` # bytearray → mutable

`mv = memoryview(b"hello")` # memoryview → view of bytes (no copy)

7. None Type → no value / empty

`n = None`

Check type

`print(type(x))`

QUICK MEMORY HACK:

Numbers → int, float, complex

Text/Collection → str, list, tuple, range

Unique → set, frozenset

Key-Value → dict

Logic → bool

Raw Data → bytes, bytearray, memoryview

Empty → None

Python Numbers

There are three numeric types in Python:

- `int`
- `float`
- `complex`

Variables of numeric types are created when you assign a value to them:

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex
```

Int

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

```
x = 1
y = 35656222554887711
z = -3255522
```

Float

Float, or "floating point number" is a number, positive or negative, containing one or more decimals.

```
x = 1.10
y = 1.0
z = -35.59
```

Complex

Complex numbers are written with a "j" as the imaginary part:

```
x = 3+5j
y = 5j
z = -5j
```

Type Conversion

You can convert from one type to another with the [int\(\)](#), [float\(\)](#), and [complex\(\)](#) methods:

```
x = 1      # int
y = 2.8    # float
z = 1j     # complex

#convert from int to float:
a = float(x)

#convert from float to int:
b = int(y)

#convert from int to complex:
c = complex(x)
```

Random Number

Python does not have a `random()` function to make a random number, but Python has a built-in module called [random](#) that can be used to make random numbers:

```
import random

print(random.randrange(1, 10))
```

Specify a Variable Type

There may be times when you want to specify a type on to a variable. This can be done with casting. Python is an object-orientated language, and as such it uses classes to define data types, including its primitive types.

Casting in python is therefore done using constructor functions:

- [int\(\)](#) - constructs an integer number from an integer literal, a float literal (by removing all decimals), or a string literal (providing the string represents a whole number)
- [float\(\)](#) - constructs a float number from an integer literal, a float literal or a string literal (providing the string represents a float or an integer)
- [str\(\)](#) - constructs a string from a wide variety of data types, including strings, integer literals and float literals

```
x = int(1)      # x will be 1
y = int(2.8)    # y will be 2
z = int("3")    # z will be 3

x = str("s1")   # x will be 's1'
y = str(2)      # y will be '2'
```

```
z = str(3.0) # z will be '3.0'
```

Strings

Strings in python are surrounded by either single quotation marks, or double quotation marks.

'hello' is the same as "hello".

You can display a string literal with the [print\(\)](#) function:

```
print("Hello")  
print('Hello')
```

Quotes Inside Quotes

You can use quotes inside a string, as long as they don't match the quotes surrounding the string:

```
print("It's alright")  
print("He is called 'Johnny'")  
print('He is called "Johnny"')
```

Assign String to a Variable

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

Strings are Arrays

Like many other popular programming languages, strings in Python are arrays of unicode characters.

However, Python does not have a character data type, a single character is simply a string with a length of 1.

Square brackets can be used to access elements of the string.

```
a = "Hello, World!"
```

```
print(a[1])
```

Looping Through a String

Since strings are arrays, we can loop through the characters in a string, with a [for](#) loop.

```
for x in "banana":  
  
    print(x)
```

String Length

To get the length of a string, use the [len\(\)](#) function.

```
a = "Hello, World!"  
  
print(len(a))
```

Check String

To check if a certain phrase or character is present in a string, we can use the keyword [in](#).

```
txt = "The best things in life are free!"  
  
print("free" in txt)
```

Use it in an [if](#) statement:

```
txt = "The best things in life are free!"  
  
if "free" in txt:  
  
    print("Yes, 'free' is present.")
```

Check if NOT

To check if a certain phrase or character is NOT present in a string, we can use the keyword [not in](#).

```
txt = "The best things in life are free!"  
  
print("expensive" not in txt)
```

Slicing

You can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

```
b = "Hello, World!"
```

```
print(b[2:5])
```

Slice From the Start

By leaving out the start index, the range will start at the first character:

```
b = "Hello, World!"
```

```
print(b[:5])
```

Slice To the End

By leaving out the *end* index, the range will go to the end:

```
b = "Hello, World!"
```

```
print(b[2:])
```

Negative Indexing

Use negative indexes to start the slice from the end of the string:

Example

Get the characters:

From: "o" in "World!" (position -5)

To, but not included: "d" in "World!" (position -2):

```
b = "Hello, World!"
```

```
print(b[-5:-2])
```

Python has a set of built-in methods that you can use on strings.

Upper Case

```
a = "Hello, World!"
```

```
print(a.upper())
```

Lower Case

```
a = "Hello, World!"  
  
print(a.lower())
```

Remove Whitespace

Whitespace is the space before and/or after the actual text, and very often you want to remove this space.

```
a = " Hello, World! "  
  
print(a.strip()) # returns "Hello, World!"
```

Replace String

```
a = "Hello, World!"  
  
print(a.replace("H", "J"))
```

Split String

The [split\(\)](#) method returns a list where the text between the specified separator becomes the list items.

```
a = "Hello, World!"  
  
print(a.split(", ")) # returns ['Hello', ' World!']
```

String Concatenation

To concatenate, or combine, two strings you can use the + operator.

```
a = "Hello"  
  
b = "World"  
  
c = a + b  
  
print(c)
```

F-Strings

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

To specify a string as an f-string, simply put an `f` in front of the string literal, and add curly brackets `{}` as placeholders for variables and other operations.

```
age = 36

txt = f"My name is John, I am {age}"

print(txt)
```

Placeholders and Modifiers

A placeholder can contain variables, operations, functions, and modifiers to format the value.

```
price = 59

txt = f"The price is {price} dollars"

print(txt)
```

A placeholder can include a *modifier* to format the value.

A modifier is included by adding a colon `:` followed by a legal formatting type, like `.2f` which means fixed point number with 2 decimals:

```
price = 59

txt = f"The price is {price:.2f} dollars"

print(txt)
```

Escape Character

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash `\` followed by the character you want to insert.

An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

```
txt = "We are the so-called \"Vikings\" from the north."
```

PYTHON STRING METHODS (EASY THEORY + EXAMPLES)

```
text = " hello world "
```

1. Case Conversion → change letter case

```
print(text.upper())    # " HELLO WORLD "
```

```
print(text.lower())    # " hello world "
```

```
print(text.title())    # " Hello World "
```

```
print(text.capitalize()) # " hello world "
```

2. Remove Spaces → clean text

```
print(text.strip())    # "hello world"
```

```
print(text.lstrip())   # "hello world "
```

```
print(text.rstrip())   # " hello world"
```

3. Search → find position

```
print(text.find("world")) # index or -1 if not found
```

```
print(text.index("world")) # index (error if not found)
```

4. Replace → change part of string

```
print(text.replace("world", "Python")) # " hello Python "
```

5. Split & Join → break and combine

```
words = text.strip().split(" ") # ['hello', 'world']
```

```
joined = "-".join(words)        # "hello-world"
```

6. Check Methods → return True/False

```
print("abc".isalpha()) # True → only letters
```

```
print("123".isdigit()) # True → only numbers
```

```
print("abc123".isalnum()) # True → letters + numbers
```

```
print("hello".islower()) # True
```

```
print("HELLO".isupper()) # True
```

7. Starts/Ends → check beginning or end

```
print(text.strip().startswith("hello")) # True
```

```
print(text.strip().endswith("world")) # True
```

8. Count → count occurrences

```
print(text.count("o")) # number of 'o'
```

9. Formatting → insert values

```
name = "Alice"
```

```
age = 25
```

```
print(f"My name is {name} and I am {age}")
```

QUICK MEMORY HACK:

Case → upper, lower, title

Clean → strip

Search → find, index

Change → replace

Break/Join → split, join

Check → isalpha, isdigit, etc.

Boolean Values

In programming you often need to know if an expression is [True](#) or [False](#).

You can evaluate any expression in Python, and get one of two answers, [True](#) or [False](#).

When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
print(10 > 9)
```

```
print(10 == 9)
```

```
print(10 < 9)
```

```
a = 200
```

```
b = 33
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
else:
```

```
    print("b is not greater than a")
```

Evaluate Values and Variables

The [bool\(\)](#) function allows you to evaluate any value, and give you [True](#) or [False](#) in return,

```
x = "Hello"
```

```
y = 15
```

```
print(bool(x))
```

```
print(bool(y))
```

Most Values are True

Almost any value is evaluated to [True](#) if it has some sort of content.

Any string is [True](#), except empty strings.

Any number is [True](#), except 0.

Any list, tuple, set, and dictionary are [True](#), except empty ones.

```
bool("abc")
```

```
bool(123)
```

```
bool(["apple", "cherry", "banana"])
```

Some Values are False

In fact, there are not many values that evaluate to [False](#), except empty values, such as `()`, `[]`, `{}`, `""`, the number `0`, and the value `None`. And of course the value [False](#) evaluates to [False](#).

```
bool(False)
```

```
bool(None)
```

```
bool(0)
```

```
bool("")
```

```
bool(())
```

```
bool([])
```

```
bool({})
```

Functions can Return a Boolean

You can create functions that returns a Boolean Value:

```
def myFunction() :  
    return True  
  
print(myFunction())
```

Python Operators

Operators are used to perform operations on variables and values.

In the example below, we use the `+` operator to add together two values:

```
print(10 + 5)
```

Although the `+` operator is often used to add together two values, like in the example above, it can also be used to add together a variable and a value, or two variables:

```
sum1 = 100 + 50      # 150 (100 + 50)  
sum2 = sum1 + 250    # 400 (150 + 250)  
sum3 = sum2 + sum2   # 800 (400 + 400)
```

```
x = 15  
y = 4
```

```
print(x + y)  
print(x - y)  
print(x * y)  
print(x / y)  
print(x % y)  
print(x ** y)  
print(x // y)
```

Division in Python

Python has two division operators:

- / - Division (returns a float)
- // - Floor division (returns an integer)

The Walrus Operator

Python 3.8 introduced the `:=` operator, known as the "walrus operator". It assigns values to variables as part of a larger expression:

```
numbers = [1, 2, 3, 4, 5]
```

```
if (count := len(numbers)) > 3:  
    print(f"List has {count} elements")
```

Comparison Operators

Comparison operators are used to compare two values:

```
x = 5
```

```
y = 3
```

```
print(x == y)
```

```
print(x != y)
```

```
print(x > y)
```

```
print(x < y)
```

```
print(x >= y)
```

```
print(x <= y)
```

Chaining Comparison Operators

Python allows you to chain comparison operators:

```
x = 5
```

```
print(1 < x < 10)
```

```
print(1 < x and x < 10)
```

Logical Operators

Logical operators are used to combine conditional statements:

and	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	$\text{not}(x < 5 \text{ and } x < 10)$

Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example	Try it
is	Returns True if both variables are the same object	$x \text{ is } y$	
is not	Returns True if both variables are not the same object	$x \text{ is not } y$	


```
x = ["apple", "banana"]  
y = ["apple", "banana"]  
z = x
```

```
print(x is z)  
print(x is y)  
print(x == y)
```

Difference Between `is` and `==`

- **is** - Checks if both variables point to the same object in memory
- **==** - Checks if the values of both variables are equal

```
x = [1, 2, 3]  
y = [1, 2, 3]  
  
print(x == y)  
print(x is y)
```

Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y

not in

Returns True if a sequence with the specified value is not present in the object

x not in y

```
fruits = ["apple", "banana", "cherry"]
```

```
print("banana" in fruits)
```

Membership in Strings

The membership operators also work with strings:

```
text = "Hello World"
```

```
print("H" in text)
```

```
print("hello" in text)
```

```
print("z" not in text)
```

Bitwise Operators

Bitwise operators are used to compare (binary) numbers:

& AND

Sets each bit to 1 if both bits are 1

x & y

| OR

Sets each bit to 1 if one of two bits is 1

x | y

^ XOR

Sets each bit to 1 if only one of two bits is 1

x ^ y

~ NOT	Inverts all the bits	~x
< Zero fill < left shift	Shift left by pushing zeros in from the right and let the leftmost bits fall off	x << 2
> Signed > right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off	x >> 2

The & operator compares each bit and set it to 1 if both are 1, otherwise it is set to 0:

```
print(6 & 3)
```

The binary representation of 6 is 0110

The binary representation of 3 is 0011

Then the & operator compares the bits and returns 0010, which is 2 in decimal.

The | operator compares each bit and set it to 1 if one or both is 1, otherwise it is set to 0:

```
print(6 | 3)
```

The binary representation of 6 is 0110

The binary representation of 3 is 0011

Then the | operator compares the bits and returns 0111, which is 7 in decimal.

The ^ operator compares each bit and set it to 1 if only one is 1, otherwise (if both are 1 or both are 0) it is set to 0:

```
print(6 ^ 3)
```

The binary representation of 6 is 0110

The binary representation of 3 is 0011

Then the ^ operator compares the bits and returns 0101, which is 5 in decimal.

Operator Precedence

Operator precedence describes the order in which operations are performed.

```
print((6 + 3) - (6 + 3))
```

Multiplication `*` has higher precedence than addition `+`, and therefore multiplications are evaluated before additions:

`()`

Parentheses

`**`

Exponentiation

`+x -x ~x`

Unary plus, unary minus, and bitwise NOT

`* / // %`

Multiplication, division, floor division, and modulus

`+ -`

Addition and subtraction

`<< >>`

Bitwise left and right shifts

`&`

Bitwise AND

`^`

Bitwise XOR

`|`

Bitwise OR

`== != > >= < <= is is not in
not in`

Comparisons, identity, and membership operators

not

Logical NOT

and

AND

or

OR

List

Lists are used to store multiple items in a single variable.

Lists are one of 4 built-in data types in Python used to store collections of data, the other 3 are [Tuple](#), [Set](#), and [Dictionary](#), all with different qualities and usage.

Lists are created using square brackets:

```
thislist = ["apple", "banana", "cherry"]  
  
print(thislist)
```

List Items

List items are ordered, changeable, and allow duplicate values.

List items are indexed, the first item has index `[0]`, the second item has index `[1]` etc.

Ordered

When we say that lists are ordered, it means that the items have a defined order, and that order will not change.

If you add new items to a list, the new items will be placed at the end of the list.

Changeable

The list is changeable, meaning that we can change, add, and remove items in a list after it has been created.

Allow Duplicates

Since lists are indexed, lists can have items with the same value:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]  
  
print(thislist)
```

List Length

To determine how many items a list has, use the [len\(\)](#) function:

```
thislist = ["apple", "banana", "cherry"]  
  
print(len(thislist))
```

List Items - Data Types

List items can be of any data type:

```
list1 = ["apple", "banana", "cherry"]  
  
list2 = [1, 5, 7, 9, 3]  
  
list3 = [True, False, False]
```

type()

From Python's perspective, lists are defined as objects with the data type 'list':

```
mylist = ["apple", "banana", "cherry"]  
  
print(type(mylist))
```

The list() Constructor

It is also possible to use the [list\(\)](#) constructor when creating a new list.

```
thislist = list(("apple", "banana", "cherry")) # note the double  
round-brackets  
  
print(thislist)
```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

Set *items* are unchangeable, but you can remove and/or add items whenever you like.

**As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and it could mean an increase in efficiency or security.

Access Items

List items are indexed and you can access them by referring to the index number:

```
thislist = ["apple", "banana", "cherry"]  
  
print(thislist[1])
```

Note: The first item has index 0.

Negative Indexing

Negative indexing means start from the end

-1 refers to the last item, -2 refers to the second last item etc.

```
thislist = ["apple", "banana", "cherry"]  
  
print(thislist[-1])
```

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new list with the specified items.

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi",  
"melon", "mango"]
```

```
print(thislist[2:5])
```

This example returns the items from the beginning to, but NOT including, "kiwi":

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi",  
"melon", "mango"]
```

```
print(thislist[:4])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the list:

This example returns the items from "orange" (-4) to, but NOT including "mango" (-1)

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi",  
"melon", "mango"]
```

```
print(thislist[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a list use the [in](#) keyword:

```
thislist = ["apple", "banana", "cherry"]
```

```
if "apple" in thislist:
```

```
    print("Yes, 'apple' is in the fruits list")
```

Change Item Value

To change the value of a specific item, refer to the index number:

```
thislist = ["apple", "banana", "cherry"]
```

```
thislist[1] = "blackcurrant"
```

```
print(thislist)
```

Change a Range of Item Values

To change the value of items within a specific range, define a list with the new values, and refer to the range of index numbers where you want to insert the new values:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi",  
"mango"]  
  
thislist[1:3] = ["blackcurrant", "watermelon"]  
  
print(thislist)
```

If you insert *more* items than you replace, the new items will be inserted where you specified, and the remaining items will move accordingly:

```
thislist = ["apple", "banana", "cherry"]  
  
thislist[1:3] = ["watermelon"]  
  
print(thislist)
```

Insert Items

To insert a new list item, without replacing any of the existing values, we can use the [insert\(\)](#) method.

The [insert\(\)](#) method inserts an item at the specified index:

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.insert(2, "watermelon")  
  
print(thislist)
```

Append Items

To add an item to the end of the list, use the [append\(\)](#) method:

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.append("orange")  
  
print(thislist)
```

Insert Items

To insert a list item at a specified index, use the [insert\(\)](#) method.

The [`insert\(\)`](#) method inserts an item at the specified index:

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.insert(1, "orange")  
  
print(thislist)
```

Extend List

To append elements from *another list* to the current list, use the [`extend\(\)`](#) method.

```
thislist = ["apple", "banana", "cherry"]  
  
tropical = ["mango", "pineapple", "papaya"]  
  
thislist.extend(tropical)  
  
print(thislist)
```

The elements will be added to the *end* of the list.

Add Any Iterable

The [`extend\(\)`](#) method does not have to append *lists*, you can add any iterable object (tuples, sets, dictionaries etc.).

```
thislist = ["apple", "banana", "cherry"]  
  
thistuple = ("kiwi", "orange")  
  
thislist.extend(thistuple)  
  
print(thislist)
```

Remove Specified Item

The [`remove\(\)`](#) method removes the specified item.

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.remove("banana")  
  
print(thislist)
```

If there are more than one item with the specified value, the [`remove\(\)`](#) method removes the first occurrence:

```
thislist = ["apple", "banana", "cherry", "banana", "kiwi"]  
  
thislist.remove("banana")  
  
print(thislist)
```

Remove Specified Index

The [`pop\(\)`](#) method removes the specified index.

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.pop(1)  
  
print(thislist)
```

If you do not specify the index, the [`pop\(\)`](#) method removes the last item.

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.pop()  
  
print(thislist)
```

The [`del`](#) keyword also removes the specified index:

```
thislist = ["apple", "banana", "cherry"]  
  
del thislist[0]  
  
print(thislist)
```

The [`del`](#) keyword can also delete the list completely.

```
thislist = ["apple", "banana", "cherry"]  
  
del thislist
```

Clear the List

The [`clear\(\)`](#) method empties the list.

The list still remains, but it has no content.

```
thislist = ["apple", "banana", "cherry"]  
  
thislist.clear()
```

```
print(thislist)
```

Loop Through a List

You can loop through the list items by using a [for](#) loop:

```
thislist = ["apple", "banana", "cherry"]  
  
for x in thislist:  
  
    print(x)
```

Loop Through the Index Numbers

You can also loop through the list items by referring to their index number.

Use the [range\(\)](#) and [len\(\)](#) functions to create a suitable iterable.

```
thislist = ["apple", "banana", "cherry"]  
  
for i in range(len(thislist)):  
  
    print(thislist[i])
```

The iterable created in the example above is `[0, 1, 2]`.

Using a While Loop

You can loop through the list items by using a [while](#) loop.

Use the [len\(\)](#) function to determine the length of the list, then start at 0 and loop your way through the list items by referring to their indexes.

Remember to increase the index by 1 after each iteration.

```
thislist = ["apple", "banana", "cherry"]  
  
i = 0  
  
while i < len(thislist):  
  
    print(thislist[i])  
  
    i = i + 1
```

Looping Using List Comprehension

List Comprehension offers the shortest syntax for looping through lists:

```
thislist = ["apple", "banana", "cherry"]
```

```
[print(x) for x in thislist]
```

- **for loop** → use when you **know how many times** you want to repeat (fixed or based on a collection like list, string, range)
- **while loop** → use when you **don't know how many times** (depends on a condition being true)

List Comprehension

List comprehension offers a shorter syntax when you want to create a new list based on the values of an existing list.

Example:

Based on a list of fruits, you want a new list, containing only the fruits with the letter "a" in the name.

Without list comprehension you will have to write a [for](#) statement with a conditional test inside:

```
[new_value for item in list]
```

```
newlist = [expression for item in iterable if condition == True]
```

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]
```

```
newlist = []
```

```
for x in fruits:
```

```
    if "a" in x:
```

```
        newlist.append(x)
```

```
print(newlist)
```

Condition

The *condition* is like a filter that only accepts the items that evaluate to [True](#).

```
newlist = [x for x in fruits if x != "apple"]
```

The condition `if x != "apple"` will return `True` for all elements other than "apple", making the new list contain all fruits except "apple".

The *condition* is optional and can be omitted:

```
newlist = [x for x in fruits]
```

Sort List Alphanumerically

List objects have a `sort()` method that will sort the list alphanumerically, ascending, by default:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]  
  
thislist.sort()  
  
print(thislist)
```

Sort Descending

To sort descending, use the keyword argument `reverse = True`:

```
thislist = ["orange", "mango", "kiwi", "pineapple", "banana"]  
  
thislist.sort(reverse = True)  
  
print(thislist)
```

```
thislist = [100, 50, 65, 82, 23]  
  
thislist.sort(reverse = True)  
  
print(thislist)
```

Customize Sort Function

You can also customize your own function by using the keyword argument `key = function`.

The function will return a number that will be used to sort the list (the lowest number first):

```
def myfunc(n):  
    return abs(n - 50)  
  
thislist = [100, 50, 65, 82, 23]  
  
thislist.sort(key = myfunc)  
  
print(thislist)
```

Case Insensitive Sort

By default the [sort\(\)](#) method is case sensitive, resulting in all capital letters being sorted before lower case letters:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
  
thislist.sort()  
  
print(thislist)
```

Luckily we can use built-in functions as key functions when sorting a list.

So if you want a case-insensitive sort function, use `str.lower` as a key function:

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
  
thislist.sort(key = str.lower)  
  
print(thislist)
```

Reverse Order

What if you want to reverse the order of a list, regardless of the alphabet?

The [reverse\(\)](#) method reverses the current sorting order of the elements.

```
thislist = ["banana", "Orange", "Kiwi", "cherry"]  
  
thislist.reverse()  
  
print(thislist)
```

Copy a List

You cannot copy a list simply by typing `list2 = list1`, because: `list2` will only be a *reference* to `list1`, and changes made in `list1` will automatically also be made in `list2`.

Use the copy() method

You can use the built-in List method [`copy\(\)`](#) to copy a list.

```
thislist = ["apple", "banana", "cherry"]  
  
mylist = thislist.copy()  
  
print(mylist)
```

Use the list() method

Another way to make a copy is to use the built-in method [`list\(\)`](#).

```
thislist = ["apple", "banana", "cherry"]  
  
mylist = list(thislist)  
  
print(mylist)
```

Use the slice Operator

You can also make a copy of a list by using the `:` (slice) operator.

```
thislist = ["apple", "banana", "cherry"]  
  
mylist = thislist[:]  
  
print(mylist)
```

Join Two Lists

There are several ways to join, or concatenate, two or more lists in Python.

One of the easiest ways are by using the `+` operator.

```
list1 = ["a", "b", "c"]  
  
list2 = [1, 2, 3]
```



```
list3 = list1 + list2
```

```
print(list3)
```

Another way to join two lists is by appending all the items from list2 into list1, one by one:

```
list1 = ["a", "b" , "c"]
```

```
list2 = [1, 2, 3]
```

```
for x in list2:
```

```
    list1.append(x)
```

```
print(list1)
```

Or you can use the [extend\(\)](#) method, where the purpose is to add elements from one list to another list:

```
list1 = ["a", "b" , "c"]
```

```
list2 = [1, 2, 3]
```

```
list1.extend(list2)
```

```
print(list1)
```

List Methods

Python has a set of built-in methods that you can use on lists.

Method	Description
<u>append()</u>	Adds an element at the end of the list

<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

Tuple

Tuples are used to store multiple items in a single variable.

Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are [List](#), [Set](#), and [Dictionary](#), all with different qualities and usage.

A tuple is a collection which is ordered and **unchangeable**.

Tuples are written with round brackets.

```
thistuple = ("apple", "banana", "cherry")  
  
print(thistuple)
```

Tuple Items

Tuple items are ordered, unchangeable, and allow duplicate values.

Tuple items are indexed, the first item has index `[0]`, the second item has index `[1]` etc.

Ordered

When we say that tuples are ordered, it means that the items have a defined order, and that order will not change.

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Allow Duplicates

Since tuples are indexed, they can have items with the same value:

```
thistuple = ("apple", "banana", "cherry", "apple", "cherry")  
  
print(thistuple)
```

Tuple Length

To determine how many items a tuple has, use the `len()` function:

```
thistuple = ("apple", "banana", "cherry")  
  
print(len(thistuple))
```

Create Tuple With One Item

To create a tuple with only one item, you have to add a comma after the item, otherwise Python will not recognize it as a tuple.

```
thistuple = ("apple",)

print(type(thistuple))
```

```
#NOT a tuple
```

```
thistuple = ("apple")

print(type(thistuple))
```

Tuple Items - Data Types

Tuple items can be of any data type:

```
tuple1 = ("apple", "banana", "cherry")

tuple2 = (1, 5, 7, 9, 3)

tuple3 = (True, False, False)
```

type()

From Python's perspective, tuples are defined as objects with the data type 'tuple':

```
mytuple = ("apple", "banana", "cherry")

print(type(mytuple))
```

The tuple() Constructor

It is also possible to use the [tuple\(\)](#) constructor to make a tuple.

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.

- [Set](#) is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- [Dictionary](#) is a collection which is ordered** and changeable. No duplicate members.

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and, it could mean an increase in efficiency or security.

Access Tuple Items

You can access tuple items by referring to the index number, inside square brackets:

```
thistuple = ("apple", "banana", "cherry")  
  
print(thistuple[1])
```

Note: The first item has index 0.

Negative Indexing

Negative indexing means start from the end.

-1 refers to the last item, -2 refers to the second last item etc.

```
thistuple = ("apple", "banana", "cherry")  
  
print(thistuple[-1])
```

Range of Indexes

You can specify a range of indexes by specifying where to start and where to end the range.

When specifying a range, the return value will be a new tuple with the specified items.

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi",  
"melon", "mango")  
  
print(thistuple[2:5])
```

 The search will start at index 2 (included) and end at index 5 (not included).

This example returns the items from the beginning to, but NOT included, "kiwi":

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi",  
"melon", "mango")
```

```
print(thistuple[:4])
```

Range of Negative Indexes

Specify negative indexes if you want to start the search from the end of the tuple:

```
thistuple = ("apple", "banana", "cherry", "orange", "kiwi",  
"melon", "mango")
```

```
print(thistuple[-4:-1])
```

Check if Item Exists

To determine if a specified item is present in a tuple use the `in` keyword:

```
thistuple = ("apple", "banana", "cherry")  
  
if "apple" in thistuple:  
  
    print("Yes, 'apple' is in the fruits tuple")
```

Tuples are unchangeable, meaning that you cannot change, add, or remove items once the tuple is created.

But there are some workarounds.

```
x = ("apple", "banana", "cherry")  
  
y = list(x)  
  
y[1] = "kiwi"  
  
x = tuple(y)
```

```
print(x)
```

Add Items

Since tuples are immutable, they do not have a built-in `append()` method, but there are other ways to add items to a tuple.

1. **Convert into a list:** Just like the workaround for *changing* a tuple, you can convert it into a list, add your item(s), and convert it back into a tuple.

```
thistuple = ("apple", "banana", "cherry")
```

```
y = list(thistuple)
y.append("orange")
thistuple = tuple(y)
```

Add tuple to a tuple. You are allowed to add tuples to tuples, so if you want to add one item, (or many), create a new tuple with the item(s), and add it to the existing tuple:

```
thistuple = ("apple", "banana", "cherry")
y = ("orange",)
thistuple += y

print(thistuple)
```

```
thistuple = ("apple", "banana", "cherry")
```

```
del thistuple
```

```
print(thistuple) #this will raise an error because the tuple no longer exists
```

Unpacking a Tuple

When we create a tuple, we normally assign values to it. This is called "packing" a tuple:

But, in Python, we are also allowed to extract the values back into variables. This is called "unpacking":

```
fruits = ("apple", "banana", "cherry")

(green, yellow, red) = fruits

print(green)
```

```
print(yellow)
```

```
print(red)
```

Note: The number of variables must match the number of values in the tuple, if not, you must use an asterisk to collect the remaining values as a list.

Using Asterisk*

If the number of variables is less than the number of values, you can add an `*` to the variable name and the values will be assigned to the variable as a list:

```
fruits = ("apple", "banana", "cherry", "strawberry", "raspberry")
```

```
(green, yellow, *red) = fruits
```

```
print(green)
```

```
print(yellow)
```

```
print(red)
```

If the asterisk is added to another variable name than the last, Python will assign values to the variable until the number of values left matches the number of variables left.

```
fruits = ("apple", "mango", "papaya", "pineapple", "cherry")
```

```
(green, *tropic, red) = fruits
```

```
print(green)
```

```
print(tropic)
```

```
print(red)
```

Loop Through a Tuple

You can loop through the tuple items by using a [for](#) loop.


```
thistuple = ("apple", "banana", "cherry")  
  
for x in thistuple:  
  
    print(x)
```

Loop Through the Index Numbers

You can also loop through the tuple items by referring to their index number.

Use the [range\(\)](#) and [len\(\)](#) functions to create a suitable iterable.

```
thistuple = ("apple", "banana", "cherry")  
  
for i in range(len(thistuple)):  
  
    print(thistuple[i])
```

Using a While Loop

You can loop through the tuple items by using a [while](#) loop.

Use the [len\(\)](#) function to determine the length of the tuple, then start at 0 and loop your way through the tuple items by referring to their indexes.

Remember to increase the index by 1 after each iteration.

```
thistuple = ("apple", "banana", "cherry")  
  
i = 0  
  
while i < len(thistuple):  
  
    print(thistuple[i])  
  
    i = i + 1
```

Join Two Tuples

To join two or more tuples you can use the `+` operator:

```
tuple1 = ("a", "b" , "c")  
  
tuple2 = (1, 2, 3)
```

```
tuple3 = tuple1 + tuple2
```

```
print(tuple3)
```

Multiply Tuples

If you want to multiply the content of a tuple a given number of times, you can use the `*` operator:

```
fruits = ("apple", "banana", "cherry")
```

```
mytuple = fruits * 2
```

```
print(mytuple)
```

Tuple Methods

Python has two built-in methods that you can use on tuples.

Method	Description
<code>count()</code>	Returns the number of times a specified value occurs in a tuple
<code>index()</code>	Searches the tuple for a specified value and returns the position of where it was found

```
myset = {"apple", "banana", "cherry"}
```

Set

Sets are used to store multiple items in a single variable.

Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are [List](#), [Tuple](#), and [Dictionary](#), all with different qualities and usage.

A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.

Set *items* are unchangeable, but you can remove items and add new items. Sets are written with curly brackets.

```
thisset = {"apple", "banana", "cherry"}  
  
print(thisset)
```

Sets are unordered, so you cannot be sure in which order the items will appear.

Set Items

Set items are unordered, unchangeable, and do not allow duplicate values.

Unordered

Unordered means that the items in a set do not have a defined order.

Set items can appear in a different order every time you use them, and cannot be referred to by index or key.

Unchangeable

Set items are unchangeable, meaning that we cannot change the items after the set has been created.

Once a set is created, you cannot change its items, but you can remove items and add new items.

Duplicates Not Allowed

Sets cannot have two items with the same value.

```
thisset = {"apple", "banana", "cherry", "apple"}  
  
print(thisset)
```

The values `True` and `1` are considered the same value in sets, and are treated as duplicates:

The values `False` and `0` are considered the same value in sets, and are treated as duplicates:

```
thisset = {"apple", "banana", "cherry", False, True, 0}
```

```
print(thisset)
```

Get the Length of a Set

To determine how many items a set has, use the [len\(\)](#) function.

```
thisset = {"apple", "banana", "cherry"}
```

```
print(len(thisset))
```

Set Items - Data Types

Set items can be of any data type:

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {1, 5, 7, 9, 3}
```

```
set3 = {True, False, False}
```

The set() Constructor

It is also possible to use the [set\(\)](#) constructor to make a set.

```
thisset = set(("apple", "banana", "cherry")) # note the double  
round-brackets
```

```
print(thisset)
```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- [List](#) is a collection which is ordered and changeable. Allows duplicate members.
- [Tuple](#) is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- [Dictionary](#) is a collection which is ordered** and changeable. No duplicate members.

When choosing a collection type, it is useful to understand the properties of that type. Choosing the right type for a particular data set could mean retention of meaning, and, it could mean an increase in efficiency or security.

Access Items

You cannot access items in a set by referring to an index or a key.

But you can loop through the set items using a [for](#) loop, or ask if a specified value is present in a set, by using the [in](#) keyword.

```
thisset = {"apple", "banana", "cherry"}
```

```
for x in thisset:
```

```
    print(x)
```

Check if "banana" is present in the set:

```
thisset = {"apple", "banana", "cherry"}
```

```
print("banana" in thisset)
```

```
thisset = {"apple", "banana", "cherry"}
```

```
print("banana" not in thisset)
```

Once a set is created, you cannot change its items, but you can add new items.

Add Items

To add one item to a set use the [add\(\)](#) method.

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.add("orange")
```

```
print(thisset)
```

Add Sets

To add items from another set into the current set, use the [update\(\)](#) method.

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}
```

```
thisset.update(tropical)
```

```
print(thisset)
```

Add Any Iterable

The object in the [update\(\)](#) method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

```
thisset = {"apple", "banana", "cherry"}  
mylist = ["kiwi", "orange"]
```

```
thisset.update(mylist)
```

```
print(thisset)
```

Remove Item

To remove an item in a set, use the [remove\(\)](#), or the [discard\(\)](#) method.

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.remove("banana")
```

```
print(thisset)
```

If the item to remove does not exist, [`remove\(\)`](#) will raise an error.

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.discard("banana")
```

```
print(thisset)
```

Note: If the item to remove does not exist, [`discard\(\)`](#) will **NOT** raise an error.

You can also use the [`pop\(\)`](#) method to remove an item, but this method will remove a random item, so you cannot be sure what item that gets removed.

The return value of the [`pop\(\)`](#) method is the removed item

```
thisset = {"apple", "banana", "cherry"}
```

```
x = thisset.pop()
```

```
print(x)
```

```
print(thisset)
```

Sets are *unordered*, so when using the `pop()` method, you do not know which item that gets removed.

```
thisset = {"apple", "banana", "cherry"}
```

```
thisset.clear()
```

```
print(thisset)
```

The `del` keyword will delete the set completely:

```
thisset = {"apple", "banana", "cherry"}
```

```
del thisset
```

```
print(thisset)
```

Loop Items

You can loop through the set items by using a `for` loop:

```
thisset = {"apple", "banana", "cherry"}
```

```
for x in thisset:
```

```
    print(x)
```

Join Sets

There are several ways to join two or more sets in Python.

The [`union\(\)`](#) and [`update\(\)`](#) methods joins all items from both sets.

The [`intersection\(\)`](#) method keeps ONLY the duplicates.

The [`difference\(\)`](#) method keeps the items from the first set that are not in the other set(s).

The [`symmetric\_difference\(\)`](#) method keeps all items EXCEPT the duplicates.

Union

The [`union\(\)`](#) method returns a new set with all items from both sets.

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1.union(set2)
```

```
print(set3)
```

You can use the `|` operator instead of the [`union\(\)`](#) method, and you will get the same result.

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set3 = set1 | set2
```

```
print(set3)
```

Join Multiple Sets

All the joining methods and operators can be used to join multiple sets.

When using a method, just add more sets in the parentheses, separated by commas:

```
set1 = {"a", "b", "c"}

set2 = {1, 2, 3}

set3 = {"John", "Elena"}

set4 = {"apple", "bananas", "cherry"}

myset = set1.union(set2, set3, set4)

print(myset)
```

When using the `|` operator, separate the sets with more `|` operators:

```
set1 = {"a", "b", "c"}

set2 = {1, 2, 3}

set3 = {"John", "Elena"}

set4 = {"apple", "bananas", "cherry"}
```

```
myset = set1 | set2 | set3 | set4
```

```
print(myset)
```

join a Set and a Tuple

The [union\(\)](#) method allows you to join a set with other data types, like lists or tuples.

The result will be a set.

```
x = {"a", "b", "c"}
```

```
y = (1, 2, 3)
```

```
z = x.union(y)
```

```
print(z)
```

Note: The `|` operator only allows you to join sets with sets, and not with other data types like you can with the [union\(\)](#) method.

Update

The [update\(\)](#) method inserts all items from one set into another.

The [update\(\)](#) changes the original set, and does not return a new set.

```
set1 = {"a", "b", "c"}
```

```
set2 = {1, 2, 3}
```

```
set1.update(set2)
```

```
print(set1)
```

Note: Both `union()` and `update()` will exclude any duplicate items.

Intersection

Keep ONLY the duplicates

The `intersection()` method will return a new set, that only contains the items that are present in both sets.

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {"google", "microsoft", "apple"}
```

```
set3 = set1.intersection(set2)
```

```
print(set3)
```

You can use the `&` operator instead of the `intersection()` method, and you will get the same result.

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {"google", "microsoft", "apple"}
```

```
set3 = set1 & set2
```

```
print(set3)
```

Note: The `&` operator only allows you to join sets with sets, and not with other data types like you can with the [intersection\(\)](#) method.

The [intersection_update\(\)](#) method will also keep ONLY the duplicates, but it will change the original set instead of returning a new set.

```
set1 = {"apple", "banana", "cherry"}

set2 = {"google", "microsoft", "apple"}

set1.intersection_update(set2)
```

```
print(set1)
```

The values [True](#) and [1](#) are considered the same value. The same goes for [False](#) and [0](#).

```
set1 = {"apple", 1, "banana", 0, "cherry"}

set2 = {False, "google", 1, "apple", 2, True}

set3 = set1.intersection(set2)
```

```
print(set3)
```

Difference

The [`difference\(\)`](#) method will return a new set that will contain only the items from the first set that are not present in the other set.

```
set1 = {"apple", "banana", "cherry"}

set2 = {"google", "microsoft", "apple"}

set3 = set1.difference(set2)

print(set3)
```

You can use the `-` operator instead of the [`difference\(\)`](#) method, and you will get the same result.

```
set1 = {"apple", "banana", "cherry"}

set2 = {"google", "microsoft", "apple"}

set3 = set1 - set2

print(set3)
```

Note: The `-` operator only allows you to join sets with sets, and not with other data types like you can with the [`difference\(\)`](#) method.

The [`difference\_update\(\)`](#) method will keep the items from the first set that are not in the other set, but it will change the original set instead of returning a new set.

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {"google", "microsoft", "apple"}
```

```
set1.difference_update(set2)
```

```
print(set1)
```

Symmetric Differences

The [`symmetric\_difference\(\)`](#) method will keep only the elements that are NOT present in both sets.

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {"google", "microsoft", "apple"}
```

```
set3 = set1.symmetric_difference(set2)
```

```
print(set3)
```

You can use the `^` operator instead of the [`symmetric\_difference\(\)`](#) method, and you will get the same result.

```
set1 = {"apple", "banana", "cherry"}
```

```
set2 = {"google", "microsoft", "apple"}
```

```
set3 = set1 ^ set2
```

```
print(set3)
```

Note: The `^` operator only allows you to join sets with sets, and not with other data types like you can with the [`symmetric\_difference\(\)`](#) method.

The `symmetric_difference_update()` method will also keep all but the duplicates, but it will change the original set instead of returning a new set.

```
set1 = {"apple", "banana", "cherry"}

set2 = {"google", "microsoft", "apple"}

set1.symmetric_difference_update(set2)

print(set1)
```

Python frozenset

`frozenset` is an immutable version of a set.

Like sets, it contains unique, unordered, unchangeable elements.

Unlike sets, elements cannot be added or removed from a frozenset.

Creating a frozenset

Use the `frozenset()` constructor to create a frozenset from any iterable.

```
x = frozenset({"apple", "banana", "cherry"})

print(x)

print(type(x))
```


Frozenset Methods

Being immutable means you cannot add or remove elements. However, frozensets support all non-mutating operations of sets.

Method	Shortcut	Description
<code>copy()</code>		Returns a shallow copy
<code>difference()</code>	<code>-</code>	Returns a new frozenset with the difference
<code>intersection()</code>	<code>&</code>	Returns a new frozenset with the intersection
<code>isdisjoint()</code>		Returns whether two frozensets have an intersection
<code>issubset()</code>	<code><= / <</code>	Returns True if this frozenset is a (proper) subset of another
<code>issuperset()</code>	<code>>= / ></code>	Returns True if this frozenset is a (proper) superset of another
<code>symmetric_difference()</code>	<code>^</code>	Returns a new frozenset with the symmetric differences
<code>union()</code>	<code> </code>	Returns a new frozenset containing the union

Set Methods

Python has a set of built-in methods that you can use on sets.

Method	Shortcut	Description
<code>add()</code>		Adds an element to the set
<code>clear()</code>		Removes all the elements from the set
<code>copy()</code>		Returns a copy of the set
<code>difference()</code>	-	Returns a set containing the difference between two or more sets
<code>difference_update()</code>	--=	Removes the items in this set that are also included in another, specified set
<code>discard()</code>		Remove the specified item
<code>intersection()</code>	&	Returns a set, that is the intersection of two other sets
<code>intersection_update()</code>	&=	Removes the items in this set that are not present in other, specified set(s)
<code>isdisjoint()</code>		Returns whether two sets have a intersection or not

<u>issubset()</u>	$\leq$	Returns True if all items of this set is present in another set
	$<$	Returns True if all items of this set is present in another, <i>larger</i> set
<u>issuperset()</u>	$\geq$	Returns True if all items of another set is present in this set
	$>$	Returns True if all items of another, <i>smaller</i> set is present in this set
<u>pop()</u>		Removes an element from the set
<u>remove()</u>		Removes the specified element
<u>symmetric_difference()</u>	$\wedge$	Returns a set with the symmetric differences of two sets
<u>symmetric_difference_update()</u>	$\wedge=$	Inserts the symmetric differences from this set and another
<u>union()</u>	$ $	Return a set containing the union of sets
<u>update()</u>	$ =$	Update the set with the union of this set and others

Python Dictionaries

```
thisdict = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
  
}
```

Dictionary

Dictionaries are used to store data values in key:value pairs.

A dictionary is a collection which is ordered*, changeable and do not allow duplicates.

As of Python version 3.7, dictionaries are *ordered*. In Python 3.6 and earlier, dictionaries are *unordered*.

```
thisdict = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964
```

```
}
```

```
print(thisdict)
```

Dictionary Items

Dictionary items are ordered, changeable, and do not allow duplicates.

Dictionary items are presented in key:value pairs, and can be referred to by using the key name.

```
thisdict = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
  
}
```

```
print(thisdict["brand"])
```

Ordered or Unordered?

When we say that dictionaries are ordered, it means that the items have a defined order, and that order will not change.

Unordered means that the items do not have a defined order, you cannot refer to an item by using an index.

Changeable

Dictionaries are changeable, meaning that we can change, add or remove items after the dictionary has been created.

Duplicates Not Allowed

Dictionaries cannot have two items with the same key:

```
thisdict = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964,  
  
    "year": 2020  
  
}  
  
print(thisdict)
```

Dictionary Length

To determine how many items a dictionary has, use the [len\(\)](#) function:

```
print(len(thisdict))
```

Dictionary Items - Data Types

The values in dictionary items can be of any data type:

```
thisdict = {  
  
    "brand": "Ford",
```

```

"electric": False,

"year": 1964,

"colors": ["red", "white", "blue"]

}

```

The dict() Constructor

It is also possible to use the [dict\(\)](#) constructor to make a dictionary.

```

thisdict = dict(name = "John", age = 36, country = "Norway")

print(thisdict)

```

Python Collections (Arrays)

There are four collection data types in the Python programming language:

- **List** is a collection which is ordered and changeable. Allows duplicate members.
- **Tuple** is a collection which is ordered and unchangeable. Allows duplicate members.
- **Set** is a collection which is unordered, unchangeable*, and unindexed. No duplicate members.
- **Dictionary** is a collection which is ordered** and changeable. No duplicate members.

Feature	List	Tuple	Set	Dictionary
Syntax	[]	()	{ }	{key: value}
Ordered?	✓ Yes	✓ Yes	✗ No	✓ Yes (Python 3.7+)

Mutable?	✓ Yes	✗ No	✓ Yes	✓ Yes
Allows Duplicates?	✓ Yes	✓ Yes	✗ No	✗ Keys: No (Values: Yes)
Indexing?	✓ Yes	✓ Yes	✗ No	✗ (Use keys instead)
Access Method	<code>list[i]</code>	<code>tuple[i]</code>	✗ Not possible	<code>dict[key]</code>
Use Case	General collection	Fixed data	Unique items	Key-value mapping

Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets:

```
thisdict = {

    "brand": "Ford",

    "model": "Mustang",

    "year": 1964

}
```

```
x = thisdict["model"]
```

There is also a method called [`get\(\)`](#) that will give you the same result:

```
x = thisdict.get("model")
```


Get Keys

The `keys()` method will return a list of all the keys in the dictionary.

```
x = thisdict.keys()
```

```
car = {
```

```
    "brand": "Ford",
```

```
    "model": "Mustang",
```

```
    "year": 1964
```

```
}
```

```
x = car.keys()
```

```
print(x) #before the change
```

```
car["color"] = "white"
```

```
print(x) #after the change
```

Get Values

The [`values\(\)`](#) method will return a list of all the values in the dictionary.

```
x = thisdict.values()
```

The list of the values is a *view* of the dictionary, meaning that any changes done to the dictionary will be reflected in the values list.

```
car = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
}
```

```
x = car.values()
```

```
print(x) #before the change
```

```
car["year"] = 2020
```

```
print(x) #after the change
```

Add a new item to the original dictionary, and see that the values list gets updated as well:

```
car = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
  
}
```

```
x = car.values()
```

```
print(x) #before the change
```

```
car["color"] = "red"
```

```
print(x) #after the change
```

Get Items

The `items()` method will return each item in a dictionary, as tuples in a list.

```
x = thisdict.items()
```

The returned list is a *view* of the items of the dictionary, meaning that any changes done to the dictionary will be reflected in the items list.

```
car = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
  
}
```

```
x = car.items()
```

```
print(x) #before the change
```

```
car["year"] = 2020
```

```
print(x) #after the change
```

Check if Key Exists

To determine if a specified key is present in a dictionary use the `in` keyword:

```
thisdict = {  
  
    "brand": "Ford",  
  
    "model": "Mustang",  
  
    "year": 1964  
  
}  
  
if "model" in thisdict:  
  
    print("Yes, 'model' is one of the keys in the thisdict dictionary")
```

User Input

Python allows for user input.

That means we are able to ask the user for input.

The following example asks for your name, and when you enter a name, it gets printed on the screen:

```
print("Enter your name:")

name = input()

print(f"Hello {name}")
```

Using prompt

In the example above, the user had to input their name on a new line. The Python [input\(\)](#) function has a **prompt** parameter, which acts as a message you can put in front of the user input, on the same line:

```
name = input("Enter your name:")

print(f"Hello {name}")
```

Multiple Inputs

You can add as many inputs as you want, Python will stop executing at each of them, waiting for user input:

```
name = input("Enter your name:")

print(f"Hello {name}")

fav1 = input("What is your favorite animal:")

fav2 = input("What is your favorite color:")

fav3 = input("What is your favorite number:")

print(f"Do you want a {fav2} {fav1} with {fav3} legs?")
```

Input Number

The input from the user is treated as a string. Even if, in the example above, you can input a number, the Python interpreter will still treat it as a string.

You can convert the input into a number with the [float\(\)](#) function:

```
x = input("Enter a number:")

#find the square root of the number:

y = math.sqrt(float(x))

print(f"The square root of {x} is {y}")
```

Validate Input

It is a good practice to validate any input from the user. In the example above, an error will occur if the user inputs something other than a number.

To avoid getting an error, we can test the input, and if it is not a number, the user could get a message like "Wrong input, please try again", and allowed to make a new input:

```
y = True

while y == True:

    x = input("Enter a number:")

    try:

        x = float(x);

        y = False

    except:
```

```
print("Wrong input, please try again.")
```

```
print("Thank you!")
```

Python None

`None` is a special constant in Python that represents the absence of a value.

Its data type is `NoneType`, and `None` is the only instance of a `NoneType` object.

NoneType

Variables can be assigned `None` to indicate "no value" or "not set".

```
x = None
```

```
print(x)
```

Use `type()` to see the type of a `None` value.

```
x = None
```

```
print(type(x))
```

Comparing to None

To compare a value to `None`, use the identity operator `is` or `is not`

```
result = None
```

```
if result is None:
```

```
    print("No result yet")
```

```
else:
```

```
    print("Result is ready")
```

True or False

`None` evaluates to `False` in a boolean context.

```
print(bool(None))
```

Functions returning None

Functions that do not explicitly return a value return `None` by default.

```
def myfunc():
```

```
    x = 5
```

```
x = myfunc()
```

```
print(x)
```

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

Before Python 3.6 we had to use the `format()` method.

F-Strings

F-string allows you to format selected parts of a string.

To specify a string as an f-string, simply put an `f` in front of the string literal, like this:

```
name = "Kausar"
```

```
age = 20
```

```
print(f"My name is {name} and I am {age} years old")
```

Why f-strings are AWESOME

✓ 1. Clean and readable

```
print(f"My name is {name}")
```

✓ 2. No type conversion needed

```
age = 20
```

```
print(f"Age: {age}") # works directly
```

✅ 3. You can run expressions inside {} 🤖

```
a = 5
```

```
b = 3
```

```
print(f"Sum is {a + b}")
```

✅ 4. Debugging superpower

```
x = 10
```

```
print(f"{x=}")
```

Output:

```
x=10
```

Perform Operations in F-Strings

You can perform Python operations inside the placeholders.

You can do math operations:

```
txt = f"The price is {20 * 59} dollars"
```

```
print(txt)
```

What is a Virtual Environment?

A **virtual environment** in Python is an isolated environment on your computer, where you can run and test your Python projects.

It allows you to manage project-specific dependencies without interfering with other projects or the original Python installation.

Think of a virtual environment as a separate container for each Python project. Each container:

- Has its own Python interpreter
- Has its own set of installed packages
- Is isolated from other virtual environments
- Can have different versions of the same package

Using virtual environments is important because:

- It prevents package version conflicts between projects
 - Makes projects more portable and reproducible
 - Keeps your system Python installation clean
 - Allows testing with different Python versions
-

Creating a Virtual Environment

Python has the built-in `venv` module for creating virtual environments.

To create a virtual environment on your computer, open the command prompt, and navigate to the folder where you want to create your project, then type this command:

What is a Virtual Environment?

A **virtual environment** (`venv`) is an isolated space for a Python project.

👉 It has its **own**:

- Python interpreter
- Installed libraries
- Dependencies

So it doesn't mess with other projects.

Why do we even need it?

Imagine this:

- Project A needs `numpy==1.20`
- Project B needs `numpy==1.26`



Without virtual env → conflict

With virtual env → both work perfectly



How It Works (Simple Idea)

Instead of installing packages globally:



Bad:

```
pip install flask
```



Good:

```
(inside venv) pip install flask
```

Each project gets its own isolated world 🌍



Step-by-Step Setup (Windows)



Step 1: Create a folder

```
mkdir myproject
```

```
cd myproject
```



Step 2: Create virtual environment

```
python -m venv venv
```

👉 This creates a folder called `venv/`

✅ Step 3: Activate it

```
venv\Scripts\activate
```

💡 You'll see:

```
(venv) C:\myproject>
```

That `(venv)` = you're inside it ✅

✅ Step 4: Install packages

```
pip install requests
```

Only installed inside this project 🔥

✅ Step 5: Save dependencies

```
pip freeze > requirements.txt
```

✅ Step 6: Deactivate

deactivate